



Paradigmas de Programación

| Avril Victoria Morfeo Zerbi

Programación Orientada a Objetos

Categorización de lenguajes:

- **Imperativos:** énfasis en la ejecución de instrucciones.
- **Declarativos:** énfasis en la evaluación de expresiones (Prolog).
- **Puros:** soportan un único paradigma.
- **Multiparadigma:** posibilidad de usar el estilo de programación más adecuado para cada trabajo (el más común).

Objetos:

- Entidad que tiene identidad, estado y comportamiento.
- Un sistema POO se forma de un conjunto de objetos que interactúan pasando mensajes entre sí.

Estilos:

- **Basado en clases:** todo objeto es una instancia de una clase específica (Java).
- **Basado en prototipos:** todo objeto está asociado a otro (su prototipo o padre) (JavaScript).

Estructura de definiciones:

- **Clase:** define los aspectos que comparten todos los objetos creados a partir de la clase.
- **Instancia:** un objeto creado a partir de una clase; tienen su propia identidad que permite distinguirla de otras instancias (reserva memoria para almacenar su estado).
- **Atributos:** variables de instancia que almacenan el estado de un objeto (miembros y campos).
- **Métodos:** funciones que operan sobre un objeto; determinan el comportamiento del objeto (enviar un mensaje a un objeto es mediante la invocación de uno de sus métodos).
- **Constructor:** método especial que es invocado automáticamente cuando se crea una instancia de la clase.

Unified Modeling Language (UML)

Proporciona una forma estándar de visualizar el diseño de un sistema

Convenciones:

- **Caja de clase:** Rectángulo dividido en tres secciones:
 1. Nombre de la clase (centrado y en **negrita**).
 - a. Interfaz: nombre de la interfaz en *itálica*.
 2. Atributos (tipo nombre:tipo).
 3. Métodos (formato: visibilidad + nombre + parámetros + tipo de retorno).
- **Visibilidad de miembros:**

- + público
- - privado
- # protegido
- ~ paquete

- **Relaciones**

- **Asociación**: Línea continua simple.
 - Puede incluir **multiplicidad** (número de elementos relacionados) cerca de los extremos.
 - 1 : uno
 - 0..1 : cero o uno
 - 0..* : cero o muchos
 - 1..* : uno o muchos
 - * : cero o muchos
 - Puede llevar un nombre opcional sobre la línea.
- **Agregación** (*tiene*): Línea continua con rombo vacío en el extremo de la clase contenedora.
- **Composición** (*tienen el mismo ciclo de vida*): Línea continua con rombo relleno en el extremo de la clase contenedora.
- **Generalización** (*herencia/es*): Línea continua con flecha vacía triangular hacia la clase padre.
- **Realización** (*implementa*): Línea punteada con flecha vacía triangular hacia la interfaz.
- **Dependencia** (*usa*): Línea punteada con flecha abierta hacia la clase de la que depende.

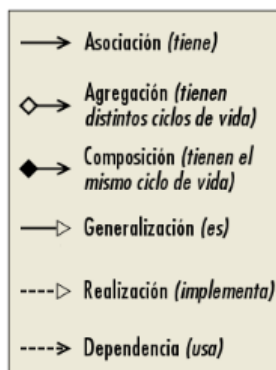
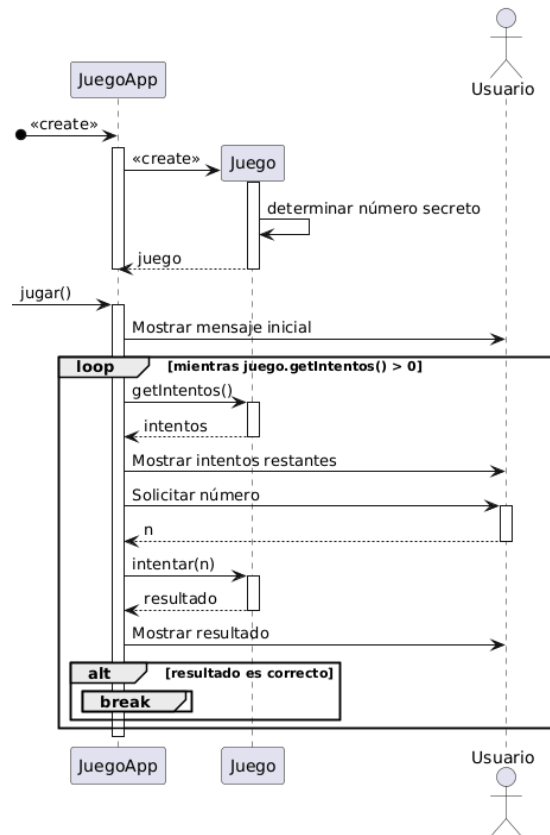


Diagrama de secuencia: muestra cómo interactúan los objetos entre sí a lo largo del tiempo para llevar a cabo un proceso o escenario específico del sistema.



- **Flecha entera:** flecha de mensaje que va de una entidad a otra entidad o la misma entidad.
- **Flecha punteada:** flecha de respuesta, es la respuesta que envía la entidad a la que se envió el mensaje.
- Los **rectángulos verticales** se llaman "caja de activación", representa el "tiempo de vida" de ejecución de una acción o método de un objeto.

Java

El código se organiza en paquetes (carpetas) y clases (un `.java` por cada clase).

Convenciones:

- Clases: `PascalCase`
- Variables, atributos, y métodos: `camelCase`
- Constantes: `UPPER_CASE`

Crear una instancia de una Clase: `new Clase(...)`

Acceder a un atributo: `instancia.atributo`

Invocar un método: `instancia.metodo(...)`

Recomendable calificar sus atributos como `private`, para evitar que puedan ser manipulados desde fuera de la clase, y opcionalmente definir métodos de acceso (getters y setters) públicos.

El modificador `final` indica que una variable o atributo es inmutable.

El modificador `static` indica que un atributo o método pertenece a la clase en sí, y no a una instancia específica.

El pasaje de **callbacks/handlers** se realiza mediante interfaces o clases abstractas.

Tipos de datos

- Primitivos: `int`, `double`, `boolean`, `char`, etc. (valor por defecto: `0`).
- Referencias: apuntan a un objeto en memoria (ej: `Punto`, valor por defecto: `null`).
- Arreglos:
 - Declaración: `int[] numeros`;
 - Inicialización: `numeros = new int[10]`; `numeros = {1, 2, 3}`;
 - Acceso: `numeros[0]`;
 - El tamaño de un arreglo es fijo una vez creado: `numeros.length`.
- Strings:
 - Son instancias de la clase `String`
 - `"..."` es azúcar sintáctica para `new String(...)`
 - Inmutables
- Colecciones dinámicas:
 - Clases disponibles en la biblioteca estándar.
 - `ArrayList`, `HashSet`, `HashMap`, etc.

Herencia y constructores

Todas las clases heredan directa o indirectamente de la clase `Object`.

Entre los métodos heredados se destaca `toString()`, que devuelve una representación en forma de cadena del objeto.

Los constructores no se heredan. Si no se define un constructor, todas las clases tienen un **constructor por defecto**: el constructor sin parámetros.

Si en el cuerpo de un constructor no hay una llamada explícita a `super()`, se invoca implícitamente como primera instrucción.

De esta manera, cuando se crea un objeto empieza una cadena de llamadas a los constructores, desde la subclase hacia arriba en la jerarquía de herencia, hasta llegar a la clase `Object`.

Si se declara un constructor con parámetros, el constructor por defecto deja de estar disponible.

Es posible sobrecargar el constructor para permitir diferentes formas de inicializar el objeto.

Scene Graph

Estructura de datos y API asociada de **JavaFX** (framework para crear aplicaciones sobre la plataforma Java) para manipular la escena.

Layout:

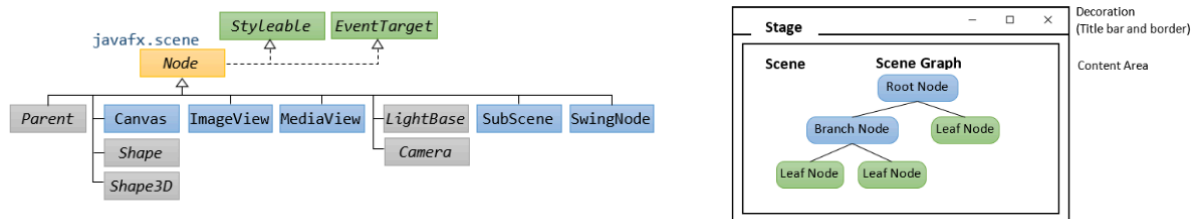
- **Stage**: representa una ventana en la pantalla. Contiene un Scene.
- **Scene**: representa el contenido de la ventana. Contiene el Scene Graph, mediante una referencia al nodo raíz.
- **Node**: Clase base para todos los nodos del Scene Graph.
- **Parent**: Clase base para los nodos que pueden tener hijos.

Controles: Son los componentes interactivos de la interfaz de usuario, como botones, etiquetas, campos de texto, etc.

Shapes: Permiten dibujar formas geométricas básicas como líneas, círculos, rectángulos, polígonos, etc.

Properties & Bindings: Clases del paquete `javafx.beans.property` ofrecen interfaces e implementaciones de properties, que encapsulan valores que pueden ser observados y enlazados (bindings).

FXML: Permite definir la estructura del Scene Graph en un archivo XML, separando la presentación de la lógica.



Programación funcional en Java

- **Interfaces funcionales:** Una interfaz funcional es aquella que tiene un único método (ej: `Predicate`, `Function`, `Consumer`, `Supplier`, etc.).
- **Funciones anónimas:** Azúcar sintáctica para crear una clase anónima que implementa una interfaz funcional.
- **Streams:** La API de Streams permite procesar secuencias de datos de manera funcional utilizando operaciones como `map`, `filter`, `reduce`, etc.
- **Record classes:** Son una forma concisa de definir clases inmutables que actúan principalmente como contenedores de datos.
- **Sealed classes:** Permiten definir jerarquías de clases con un control más estricto sobre qué clases pueden extenderse.
- **Pattern matching:** Probar si un objeto tiene una estructura determinada y luego extraer datos de ese objeto si hay coincidencia.

Hilos en Java

La clase `Thread` representa un hilo de ejecución en Java. El proceso finaliza cuando no queda ningún hilo de tipo non-daemon en ejecución.

Sincronización usando `interrupt` y `join`:

```
public class Main {
    static void mostrar(String mensaje) {
        System.out.format("%s: %s\n",
            Thread.currentThread().getName(),
            mensaje);
    }

    static void martinFierro() {
        String[] mensajes = {
            "Aquí me pongo a cantar",
            "Al compás de la vigüela",
            "Que al hombre que lo desvela",
            "Una pena extraordinaria,",
            "Como la ave solitaria",
            "Con el cantar se consuela."
        };
        try {
            for (String mensaje : mensajes) {
                Thread.sleep(4000);
                mostrar(mensaje);
            }
        } catch (InterruptedException e) {
            mostrar("No terminó!");
        }
    }
}
```

```
public static void main(String[] args) throws InterruptedException {
    long paciencia = 10_000; // milisegundos

    mostrar("Lanzando el hilo");
    long inicio = System.currentTimeMillis();
    Thread t = new Thread(Main::martinFierro);
    t.start();

    mostrar("Esperando a que el hilo termine");
    while (t.isAlive()) {
        mostrar("Sigo esperando...");
        if ((System.currentTimeMillis() - inicio) > paciencia) {
            mostrar("Me cansé de esperar!");
            t.interrupt();
        }
        t.join(1000);
    }
    mostrar("Al fin!");
}
```

La JVM define un **modelo de memoria** que especifica cómo y cuándo los cambios hechos por un hilo son visibles para otros hilos.

Todos los mecanismos de sincronización en Java se construyen en base a un **candado intrínseco** (o monitor) asociado a cada objeto.

El entrar al bloque `synchronized(candado) { ... }` el hilo intenta adquirir el candado intrínseco del objeto `candado`.

- Si el candado está libre, el hilo lo **adquiere** y entra al bloque.
- Si el candado está ocupado por otro hilo, el hilo actual se **bloquea** hasta que el candado se libere.

Al finalizar el bloque, el hilo **libera** el candado.

También es posible utilizar **candados explícitos** provistos por la biblioteca estándar. La interfaz `Lock` provee los métodos `lock()` y `unlock()`, ofreciendo un control más fino sobre la adquisición y liberación del candado.

```
public class Main {
    static int a = 0;
    static Object candado;

    static void incThread() {
        for (int i = 0; i < 100000; i++) {
            synchronized(candado) {
                a++;
            }
        }
    }

    static void decThread() {
        for (int i = 0; i < 100000; i++) {
            synchronized(candado) {
                a--;
            }
        }
    }
}
```

```
public class ContadorSincronizado {
    private int c = 0;
    private final ReentrantLock lock = new ReentrantLock();

    public void incrementar() {
        lock.lock();
        c++;
        lock.unlock();
    }

    public int valor() {
        int v;
        lock.lock();
        v = c;
        lock.unlock();
        return v;
    }
}
```

El método `wait` de la clase `Object`:

1. Libera el candado intrínseco del objeto (se asume que está adquirido).
2. Bloquea el hilo hasta que sea notificado o interrumpido desde otro hilo.
3. Cuando el hilo se despierta, vuelve a adquirir el candado intrínseco.

El método `notify` **despierta a uno** de los hilos que están esperando en el objeto (si hay alguno).

El método `notifyAll` **despierta a todos** los hilos que están esperando en el objeto (si hay alguno).

Colecciones diseñadas para ser usadas en entornos concurrentes:

- `CopyOnWriteArrayList` : Una lista que crea una copia del arreglo subyacente en cada operación de escritura. Ideal para escenarios con muchas lecturas y pocas escrituras.
- `ConcurrentHashMap` : Un mapa hash que permite acceso concurrente eficiente. Divide el mapa en segmentos para minimizar la contención entre hilos.
- `BlockingQueue` : Una cola que soporta operaciones que esperan hasta que la cola no esté vacía al retirar elementos, o hasta que haya espacio disponible al insertar elementos.

La JVM garantiza que algunas **operaciones son atómicas** (por ejemplo, la lectura y escritura de referencias y algunos tipos primitivos). La biblioteca estándar de Java provee clases en el paquete `java.util.concurrent.atomic` que permiten operaciones atómicas sobre variables, evitando la necesidad de sincronización explícita.

La interfaz `ExecutorService` define un mecanismo genérico para **gestionar** un grupo de hilos para ejecutar computaciones asíncronas.

```
long sumar(long[] array) {
    long r = 0;
    for (long v : array) {
        r += v;
    }
    return r;
}

// En cualquier momento, como máximo 4 hilos estarán activos procesando
// tareas. Si se envían tareas adicionales cuando todos los hilos están
// activos, esperarán en la cola hasta que un hilo esté disponible.
ExecutorService executor = Executors.newFixedThreadPool(4);

Future<Long> sumarAsincronico(long[] array) {
    return executor.submit(() -> sumar(array));
}

Future<Long> f = sumarAsincronico(array);
// Hacer otras cosas...
long resultado = f.get();
```

La clase `ForkJoinPool` es una implementación de la interfaz `ExecutorService` que automáticamente aplica una estrategia de divide y conquista para paralelizar tareas.

Abstracción

Se seleccionan las características más relevantes de un sistema, omitiendo los no relevantes.

Es la creación de un modelo simplificado para representar un sistema complejo, concentrándose únicamente en los conceptos relevantes para un dominio particular.

En el paradigma procedimental, la abstracción se logra principalmente mediante **funciones** y **procedimientos**.

En el paradigma de objetos, una manera de abstraer objetos es mediante la definición de **interfaces**. Otra forma de lograr la abstracción es mediante la **composición** y **agregación** de objetos.

```

public interface Lista { ... }

public class ListaEnlazada implements Lista {
    // composición: la ListaEnlazada es dueña del Nodo
    // y sus tiempos de vida están ligados.
    private Nodo primero;

    // ...
}

class Nodo {
    // agregación: el Nodo tiene una referencia a
    // otro Nodo, pero no es responsable de
    // su creación o destrucción.
    private Nodo siguiente;

    // agregación: el Nodo tiene una referencia a la
    // instancia de Object, pero no es responsable de
    // su creación o destrucción.
    private Object dato;

    // ...
}

```

Clases abstractas

Si una clase o alguno de sus métodos está marcado con la palabra clave `abstract`, entonces la clase es abstracta y no puede ser instanciada directamente.

Puede invocar uno de sus **métodos abstractos**.

Encapsulamiento

Agrupación de estado y comportamiento en una unidad conceptual (la clase).

Aplicación de mecanismos de restricción de acceso a los atributos y métodos, para ocultar los detalles internos de un objeto (modificadores de acceso: `private`, `protected` y `public`).

Definir métodos de acceso (**getters**) y métodos de modificación (**setters**) para acceder y modificar los atributos de una clase. Los setters permiten entre otras cosas validar que no se violen las invariantes del objeto.

Polimorfismo

Representar múltiples tipos de datos diferentes mediante un único símbolo.

Categorías:

- **Polimorfismo ad-hoc:** funciona con un número limitado y conocido de tipos, no necesariamente relacionados entre sí.
 - **Polimorfismo por sobrecarga**
 - Capacidad de escribir dos o más funciones o métodos con el mismo nombre pero diferente firma.
 - El resultado es un polimorfismo aparente, ya que no se trata de un método que puede recibir muchos tipos de argumentos, sino que hay varios métodos con el mismo nombre, y en tiempo de compilación se decide cuál usar según el tipo de los argumentos pasados.
 - **Polimorfismo por coerción**
 - Forzar a que un valor de un tipo sea convertido a otro tipo.
 - Se trata también de un polimorfismo aparente, ya que la conversión que ocurre no cambia el hecho de que el método, en realidad trabaja con un único tipo.
- **Polimorfismo universal:** funciona con un número ilimitado de tipos relacionados entre sí.

- **Polimorfismo paramétrico**
 - Se usa un símbolo genérico que luego puede ser sustituido por un tipo específico.
 - En Java, a esta variante de polimorfismo se la conoce como Generics.
- **Polimorfismo por inclusión**
 - Se basa en el hecho de que los tipos de datos forman una jerarquía, de forma tal que un objeto de un tipo subtipo puede ser tratado como un objeto de su tipo supertipo.

Herencia

Permite crear nuevas clases basadas en clases existentes, heredando sus atributos y métodos, y agregando o modificando funcionalidad.

A la clase previamente existente se la conoce como **superclase** y a las nuevas como **subclases**.

Herencia simple: una subclase hereda de una única superclase.

- La inexistencia de la herencia múltiple podría resultar limitante para ciertos diseños.

Herencia múltiple: una subclase puede heredar de múltiples superclases.

- El principal problema se conoce como el **problema del diamante** (¿Cómo podemos expresar que un `Celular` es al mismo tiempo un `Telefono` y un `EnviadorDeMensajes`?, ¿Y que una `PalomaMensajera` es un `Animal` y un `EnviadorDeMensajes`?). Se soluciona modelando como realizaciones todas las relaciones con `EnviadorDeMensajes` que no sería una clase sino una interfaz, esto evita el problema ya que en todos los casos hay una única implementación del método `enviar`.

Jerarquías de clases:

- Una subclase es más específica que su superclase y representa a un grupo más especializado de objetos.
- La subclase exhibe los comportamientos de su superclase junto con otros adicionales específicos de esta subclase. Es por ello que a la herencia se la conoce a veces como especialización o generalización (la flecha se lee "es un(a)").
- La **superclase directa** es la clase de la cual la subclase hereda en forma explícita.
- Una **superclase indirecta** es cualquier clase que esté arriba en la jerarquía de herencia, y de la cual la subclase hereda implícitamente.

Una subclase puede sobrescribir o redefinir cualquier método de la superclase con una implementación modificada (method **overriding**).

La palabra clave `super` permite acceder a los atributos y métodos de la superclase.

Principios de diseño

No se deben aplicar ciegamente, sino que deben ser considerados como guías para mejorar la calidad del código y la arquitectura del software. Muchos de estos principios pueden entrar en conflicto entre sí, y es importante evaluar cada situación y decidir cuál es el más adecuado para el contexto.

You Ain't Gonna Need It (YAGNI)

Solo agregar una funcionalidad si es requerida.

Keep It Simple, Stupid! (KISS)

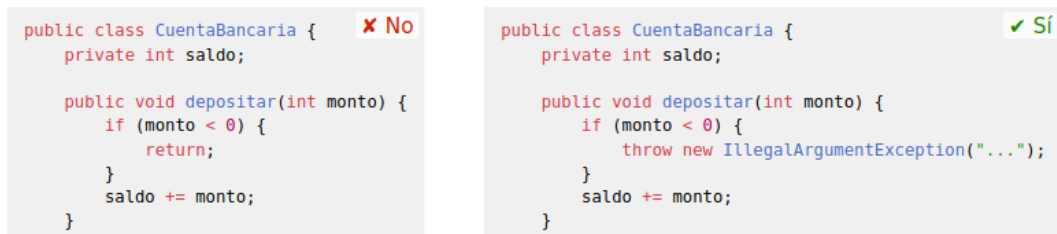
Escribir código lo más simple posible.

Don't Repeat Yourself (DRY)

No repetir código que fue implementado en otra parte (por ejemplo un método).

Principle of Least Astonishment (POLA)

Un componente de un sistema debe comportarse como la mayoría de sus usuarios esperan que se comporte, y no sorprender con comportamientos inesperados.



Knuth's Optimization Principle (KOP)

No optimizar el código prematuramente.

Separation of Concerns (SoC)

Dividir un sistema en partes independientes que abordan diferentes aspectos o dominios relacionados con el problema a resolver.

Presentación	Módulo	Módulo	Módulo
Lógica	Módulo	Módulo	Módulo
Persistencia	Módulo	Módulo	Módulo

Alta cohesión

Cohesión: es la medida en que dos elementos de un módulo están relacionados entre sí.

Es preferible que los módulos tengan alta cohesión; es decir, que sus elementos (variables, funciones, clases, métodos) estén estrechamente relacionados y trabajen juntos para cumplir un propósito específico.

Bajo acoplamiento

Acoplamiento: es la medida en que un módulo depende de otros módulos.

Es preferible que los módulos tengan bajo acoplamiento; es decir, que dependan lo menos posible de otros módulos, lo que facilita su reutilización, mantenimiento y prueba.

Tell, don't ask! (TDA)

Los objetos deben decidir por sí mismos cómo realizar sus tareas, en lugar de que otros objetos les pidan datos para hacerlo por ellos.

```

class Logger {
    public boolean habilitado;

    public void log(String message) {
        System.out.println(message);
    }
}

public class CuentaBancaria {
    private Logger logger;
    private int saldo;

    public CuentaBancaria(Logger logger) {
        this.logger = logger;
    }

    public void depositar(int monto) {
        saldo += monto;
        if (logger.habilitado) {
            logger.log("Depositando " + monto);
        }
    }
}

```

✗ No

```

class Logger {
    public boolean habilitado;

    public void log(String message) {
        if (habilitado) {
            System.out.println(message);
        }
    }
}

public class CuentaBancaria {
    private Logger logger;
    private int saldo;

    public CuentaBancaria(Logger logger) {
        this.logger = logger;
    }

    public void depositar(int monto) {
        saldo += monto;
        logger.log("Depositando " + monto);
    }
}

```

✓ Sí

Principle of Least Knowledge (PoLK)

Para promover el bajo acoplamiento, cada módulo debería conocer lo menos posible sobre otros módulos.

Aplicado a objetos, un método **f** de una clase **C** solo debería invocar métodos de:

- la propia clase **C**;
- los objetos que son atributos de **C**;
- los objetos recibidos por **f** como argumentos;
- los objetos instanciados en **f**.

```

public class CarritoDeCompras {
    private List<Item> items;

    public List<Item> getItems() {
        return items;
    }
}

public class ServicioWeb {
    public agregarAlCarrito(CarritoDeCompras carrito, Item item) {
        carrito.getItems().add(item);
    }
}

```

✗ No

```

public class CarritoDeCompras {
    private List<Item> items;

    public void agregarItem(Item item) {
        items.add(item);
    }
}

public class ServicioWeb {
    public void agregarAlCarrito(CarritoDeCompras carrito, Item item) {
        carrito.agregarItem(item);
    }
}

```

✓ Sí

Explicit Dependencies Principle (EDP)

Las clases y métodos deben requerir explícitamente los objetos necesarios para funcionar correctamente, en lugar de asumir que están disponibles en el contexto.

```

public class Logger {
    public void log(String s) {
        System.out.println(s);
    }
}

```

✗ No

```

public class Logger {
    private final PrintStream out;

    public Logger(PrintStream out) {
        this.out = out;
    }

    public void log(String s) {
        out.println(s);
    }
}

```

✓ Sí

Principio SOLID

- **S**: Single Responsibility Principle (SRP)
 - Cada clase debe tener una única responsabilidad o propósito.
- **O**: Open-Closed Principle (OCP)
 - Las clases deben estar abiertas para su extensión, pero cerradas para su modificación. Es decir, debe ser posible agregar nuevas funcionalidades a una clase sin modificar su código existente.

```

public class Enemigo {
    String tipo;

    public int getAtaque() {
        switch (tipo) {
            case "goblin": return 10;
            case "orco": return 15;
            case "dragon": return 35;
        }
    }

    public int getDefensa() {
        switch (tipo) {
            case "goblin": return 5;
            case "orco": return 10;
            case "dragon": return 20;
        }
    }
}

```

✗ No

```

public interface Enemigo {
    int getAtaque();
    int getDefensa();
}

public class Goblin implements Enemigo {
    public int getAtaque() { return 10; }
    public int getDefensa() { return 5; }
}

public class Orco implements Enemigo {
    public int getAtaque() { return 15; }
    public int getDefensa() { return 10; }
}

public class Dragon implements Enemigo {
    public int getAtaque() { return 35; }
    public int getDefensa() { return 20; }
}

```

✓ Sí

- **L: Liskov Substitution Principle (LSP)**

- Una instancia de una clase debe poder ser sustituida por una instancia de una clase derivada sin "romper" el comportamiento del programa.

```

public class Naipe {
    private String palo;
    private int numero;

    public Naipe(String palo, int numero) {
        this.palo = palo;
        this.numero = numero;
    }

    public String getPalo() { return palo; }
    public int getNumero() { return numero; }

    public String mostrar() {
        return "%s de %d".formatted(numero, palo);
    }
}

```

✗ No

```

public class Comodin extends Naipe {
    public Comodin() {
        super("Comodin", 0);
    }

    @Override public String getPalo() {
        throw new UnsupportedOperationException("Un comodín no tiene palo");
    }

    @Override public int getNumero() {
        throw new UnsupportedOperationException("Un comodín no tiene número");
    }

    public String mostrar() {
        return "Comodin";
    }
}

```

- **I: Interface Segregation Principle (ISP)**

- Una clase no debe depender de métodos de otras clases que no utiliza.

```

class Auto {
    public void llenarTanque() { ... }
    public void inflarRuedas() { ... }
    public void verMotor() { ... }
    public void usarRuedas() { ... }
    public void conducir() { ... }
}

class Mecanico {
    public void reparar(Auto auto) {
        auto.verMotor();
        auto.llenarTanque();
        auto.inflarRuedas();
    }
}

class Valet {
    public void estacionar(Auto auto) {
        auto.conducir();
    }
}

```

✗ No

```

interface Reparable {
    void verMotor();
    void llenarTanque();
    void inflarRuedas();
}

interface Conducible {
    void conducir();
}

class Auto implements Reparable, Conducible { ... }
class Mecanico {
    public void reparar(Reparable r) { ... }
}
class Valet {
    public void estacionar(Conducible c) { ... }
}

```

✓ Sí

- **D: Dependency Inversion Principle (DIP)**

- Las clases de alto nivel no deben depender de clases de bajo nivel; ambas deben depender de abstracciones.
- Las abstracciones no deben depender de detalles; los detalles deben depender de las abstracciones

```

class Auto {
    public void conducir() { ... }
}

class Valet {
    public void estacionar(Auto auto) {
        auto.conducir();
    }
}

```

✗ No

```

interface Conducible {
    void conducir();
}

class Auto implements Conducible { ... }

class Valet {
    public void estacionar(Conducible c) { ... }
}

```

✓ Sí

Composición sobre herencia

- Reutilizar código combinando objetos en lugar de heredar clases.
- Relación **"tiene un"** en lugar de **"es un"**.
- **Ventajas:**

- Menor acoplamiento entre clases.
 - Mayor flexibilidad y facilidad de mantenimiento.
 - Permite reemplazar componentes fácilmente.
 - Facilita cumplir el principio Open/Closed (SOLID).
 - **Comparación de enfoques:**
 - **Composición:**
 - Reutilización de código: ✓ Sí (por delegación)
 - Polimorfismo: ✓ Posible (si se usan interfaces)
 - Acoplamiento: ✓ Bajo
 - **Interfaces:**
 - Reutilización de código: ✗ No (solo definen contrato)
 - Polimorfismo: ✓ Sí
 - Acoplamiento: ✓ Bajo
 - **Herencia:**
 - Reutilización de código: ✓ Sí (hereda implementación)
 - Polimorfismo: ✓ Sí
 - Acoplamiento: ✗ Alto
 - La herencia tiene una estructura rígida y dependencias fuertes mientras la composición colaboración dinámica entre objetos.
 - Los patrones de diseño como Strategy y Bridge suelen combinar la composición con interfaces para lograr un diseño flexible y extensible.
-

Programación orientada a eventos

El flujo del programa es determinado por la ocurrencia de eventos que son previstos pero no planeados (es decir, no se sabe cuándo ocurrirán).

La nomenclatura varía según el framework/lenguaje:

- Objeto que produce eventos: Event source, Event emitter, Event dispatcher, Publisher
- Objeto que quiere ser notificado: Listener, Subscriber, Consumer

Polling

Esta estrategia es sincrónica: las operaciones ocurren en un **orden determinado**.

Esta estrategia se conoce también como busy waiting o busy loop: el encuestador usa recursos del sistema (CPU, energía) mientras espera a que ocurra el evento de interés.

```

long ultimo = System.currentTimeMillis();
int segundos = 0;
while (true) {
    // pasó 1 segundo desde la última vez?
    long ahora = System.currentTimeMillis();
    if (ahora - ultimo > 1000) {
        segundos += 1;
        System.out.printf("Pasaron %ds.\n", segundos);
        ultimo = ahora;
    }
}

```

Callbacks

Función que se pasa como parámetro a otra función (típicamente no bloqueante). El callback será invocado en algún momento posterior (en forma asíncrona).

Estrategia de tipo push: el emisor "empuja" los eventos a los consumidores.

Suelen ser llamados **handlers** cuando son utilizados para manejar eventos.

```

import java.util.Timer;
import java.util.TimerTask;

public class Main {
    public static void main(String[] args) {
        Timer timer = new Timer();
        Callback callback = new Callback();
        // callback.run() será invocado cada 1 segundo
        timer.scheduleAtFixedRate(callback, 0, 1000);
        // El mensaje anterior fue no bloqueante
        System.out.println("Timer configurado.");
    }
}

class Callback extends TimerTask {
    int segundos = 0;

    @Override public void run() {
        System.out.printf("Pasaron %ds.\n", segundos++);
    }
}

```

Azúcar sintáctica

Características de un lenguaje que hace que el código sea más fácil de leer o escribir sin agregar funcionalidades nuevas.

Clases anónimas

- Son clases **sin nombre**, definidas "en línea" para implementar interfaces o extender clases rápidamente.
- Reducen la verbosidad comparado con declarar una clase completa.

HttpClient y APIs

- Algunas APIs modernas están diseñadas con métodos encadenables (`.send()` , `.body()` , etc.) para que el código sea más legible.
- Esto evita el código repetitivo y hace que las operaciones parezcan más "naturales".
- Las APIs de I/O son tradicionalmente **síncronas** y **bloqueantes**. La limitación principal de esta implementación es que las funciones `.accept()` y `.read()` son bloqueantes, y por lo tanto el servidor solo puede atender a un cliente por vez.
- La estrategia de **entrada/salida multiplexada** permite bloquear hasta que ocurra algún evento de interés.

```

server = crearServerSocket()
sockets = [server]
while True:
    # bloquea hasta que ocurra algún evento de interés
    subconjunto = select(sockets)
    for socket in subconjunto:
        if type(socket) is ServerSocket:
            # aceptamos la conexión de un cliente
            cliente = server.accept()
            sockets += cliente
        else:
            # recibimos datos de un cliente
            s = socket.read()
            if not s:
                # cliente desconectado
                socket.close()
                sockets.remove(socket)
            else:
                # envía la respuesta al cliente
                socket.write(s);

```

Expresiones lambda

- Forma concisa de escribir clases anónimas para interfaces funcionales (interfaces con un solo método).
- Menos código y más legible.
- Facilita programación funcional y manejo de colecciones.

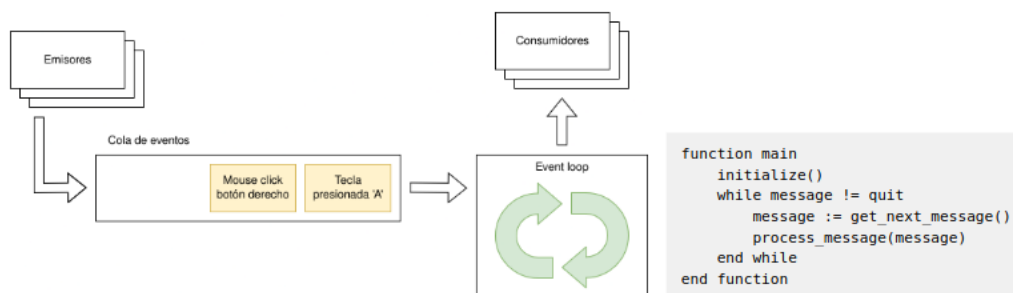
Patrón Observer

Permite registrar múltiples callbacks para ser invocados cuando ocurra un evento, de esta manera se logra una comunicación uno a muchos (un emisor, múltiples observadores).

Event loop

Estrategia que permite desacoplar aún más los emisores de los consumidores. La publicación de los eventos es **asincrónica** con la invocación de los handlers.

Ejemplo: Node.js opera en un **event loop** de un solo hilo, utilizando llamadas de I/O **no bloqueantes**, lo que le permite soportar decenas de miles de conexiones concurrentes sin incurrir en el costo del cambio de contexto entre hilos. El diseño de compartir un solo hilo entre todas las solicitudes que usan el **patrón observer** está destinado a construir aplicaciones altamente concurrentes, donde cualquier función que realice I/O debe usar un **callback**.



Interfaces gráficas

Las aplicaciones con UI suelen tener al menos 2 o 3 capas de abstracción:

- **Presentación:** se encarga de la interacción con el usuario.
- **Lógica de negocio:** se encarga de modelar el estado de la aplicación y las operaciones que pueden realizarse sobre él.
- **Persistencia:** se encarga de almacenar el estado de la aplicación de forma permanente, por ejemplo en una base de datos o un archivo.

Command Line Interface (CLI)

Suelen ser implementadas mediante un algoritmo simple: REPL. La implementación más usual es mediante un **event loop**. Ej: Terminal

Text User Interface (TUI)

interfaz en la que el usuario interactúa mediante texto, pero con una presentación más estructurada y visual que una simple línea de comandos (CLI). Se usa texto y caracteres para crear **menús, cuadros, botones o listas**, todo dentro de un entorno textual (sin gráficos). Ej: BIOS.

Graphical User Interface (GUI)

Los frameworks GUI suelen abstraer el **event loop**; el cliente solo debe escribir el código que procesa los eventos.

Existe una gran cantidad de opciones de librerías y frameworks gráficos, hay que considerar el nivel de abstracción deseado:

- **Librerías gráficas** (bajo nivel): Se encargan de dibujar píxeles en la pantalla, pero no proveen componentes de interfaz de usuario.
- **Widget toolkits:** Facilitan la creación de interfaces gráficas, con ventanas, y widgets/controles (botones, menús, etc.).
- **Application frameworks** (alto nivel): Proveen un entorno completo para desarrollar aplicaciones, incluyendo en algunos casos soporte de hardware específico como la cámara, micrófono, GPS, etc. El **event loop** suele estar abstraído en esta capa.

Modo inmediato

La escena se mantiene en la memoria de la aplicación.

La aplicación es responsable de invocar a la librería gráfica para redibujar la pantalla.

Las **librerías** gráficas de bajo nivel suelen ser de modo inmediato.

Modo retenido

La escena se mantiene en la memoria de la librería gráfica.

La librería gráfica se encarga de redibujar la pantalla en caso de ser necesario.

Los **frameworks** de alto nivel suelen ser de modo retenido.

Model View Controller (MVC)

- Modelo: lógica
- Vista: representación gráfica
- Controlador: interacción del usuario

Programación lógica (Prolog)

Se basa en expresar proposiciones, reglas y hechos lógicos, y utilizar un motor de inferencia para resolver problemas.

Es un paradigma declarativo, ya que se enfoca en qué se quiere lograr en lugar de cómo lograrlo, permitiendo que el motor de inferencia se encargue de la ejecución.

Proposición: una afirmación lógica que puede ser verdadera o falsa.

Axioma/Hecho: una proposición aceptada como verdadera sin necesidad de prueba.

Regla: una proposición que define una relación entre hechos y/o otras reglas ("Si llueve, entonces la calle está mojada").

Base de datos: un conjunto de hechos y reglas que representan el conocimiento en un dominio específico.

Objetivo/Consulta: una proposición que se desea demostrar o refutar.

Estrategia de resolución: el proceso de buscar una solución a un objetivo utilizando la base de datos.

Programa lógico: Compuesto por una base de datos y un objetivo. Al ser ejecutado, un motor de inferencia se encarga de probar o refutar el objetivo, utilizando alguna estrategia de resolución.

Tipos de datos

Prolog es dinámicamente tipado, implicando que todo valor se represente mediante un término, que tiene varios subtipos:

- **Términos atómicos:**

- Átomos: `x`, `azul`, `hola_mundo`, `'Hola, Mundo!'`.
- Números: enteros y números de punto flotante.
- Variables: `X`, `Resultado`, `_temp`.
 - Son **unificadas** con otros términos durante la resolución de objetivos.
 - La variable `_` representa una **variable anónima** y significa "cualquier término".

- **Términos compuestos:**

- Estructuras: consisten en un átomo (el **functor**) seguido de una lista de argumentos entre paréntesis (`padre(juan, maria)`, `suma(X, Y, Resultado)`).
 - La **aridad** es el número de argumentos que tiene. La notación `f/n` se usa para referirse a un functor `f` con aridad `n` (`padre/2`).
 - Algunos predicados que pueden ser escritos como operadores infijos (`=(X, Y)` es equivalente a `X = Y`).
- Listas: `[]`, `[1, 2, 3]`, `[X, Y]`
 - Si `T` es una lista, `[H|T]` es una lista con cabeza `H` y cola `T`.
- Cadenas: `"Hola, Mundo!"`

Cláusulas

Una base de datos Prolog se compone de un conjunto de cláusulas, que pueden ser reglas o hechos.

Prolog sigue una estrategia de búsqueda de tipo **depth-first**: unifica el objetivo con la primera de las alternativas y continúa la búsqueda.

El operador `:-` se lee como $\Leftarrow$, de forma que la regla se lee como "`p` entonces `q`". El encabezado de la regla es `q`, y el cuerpo es `p`.

Dos o más cláusulas con el mismo functor se consideran parte del mismo **predicado**.

La consulta se escribe como `?- t.` donde `t` es un término.

Predicados

Unidad básica de conocimiento o regla que describe hechos o relaciones entre objetos dentro del programa.

Predefinidos

- `true / false ( fail )`: Siempre evalúan a verdadero o falso, respectivamente.
- `A, B`: Verdadero si `A` y `B` son ambos verdaderos (*and*).
- `A; B`: Verdadero si `A` o `B` (o ambos) son verdaderos (*or*).
- `X == Y`: Verdadero si `X` e `Y` son términos idénticos.
- `X \== Y`: Verdadero si `X` e `Y` no son términos idénticos.
- `X = Y`: Unifica `X` e `Y` (falso si no son unificables).
- `X \= Y`: Verdadero si `X` e `Y` no son unificables.
- `var(X) / atom(X) / number(X) / ...`: Verdadero si `X` es una variable, átomo, número, etc.

De control

- `repeat`: Siempre tiene éxito, y provee un número infinito de soluciones mediante backtracking. Útil para generar bucles.
- `!`: Proposición de corte. Siempre tiene éxito de inmediato, pero no puede ser resatisfecha a través del backtracking. Por lo tanto, las submetas a su izquierda, en una meta compuesta, tampoco pueden ser resatisfechas mediante backtracking.
- `\+`: "No demostrable". Verdadero si el objetivo no puede ser satisfecho.

Unificación

Dos términos se unifican si pueden representar la misma estructura. Si alguno de los términos contiene variables, éstas se instancian para lograr la unificación.

Para resolver una consulta, el motor de Prolog:

- Intenta unificar el objetivo con alguna de las cláusulas de la base de datos (buscando de arriba hacia abajo).
- Para las cláusulas de tipo `q :- p`, el objetivo debe unificar con `q`.
- Si la cláusula unificada tiene un cuerpo `p`, se intenta resolver la consulta `?- p.`, con las variables instanciadas en la unificación de `q`.

```
• padre(homero, bart) unifica con padre(homero, bart).  
• padre(homero, bart) no unifica con padre(homero, lisa).  
• padre(homero, X) unifica con padre(homero, bart) instanciando X = bart.  
• padre(P, X) unifica con padre(homero, bart) instanciando P = homero, X = bart.  
• 2 + 3 no unifica con 5.
```

Árbol de búsqueda

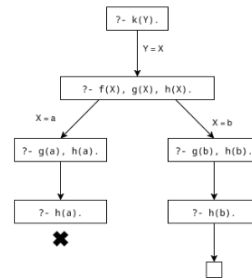
Manera de mostrar cómo Prolog resuelve una consulta.

Base de datos:

```
f(a).
f(b).
g(a).
g(b).
h(b).
k(X) :- f(X), g(X), h(X).
```

Consulta:

```
?- k(Y).
```



Modelo de seguimiento

Se puede depurar un programa en Prolog utilizando el predicado `trace`, donde la ejecución del programa se describe de manera procedimental en términos de cuatro eventos:

- **Call:** Se intenta resolver un objetivo.
- **Exit:** El objetivo se resuelve exitosamente.
- **Fail:** El objetivo no puede ser resuelto.
- **Redo:** Se reintenta resolver un objetivo para encontrar más soluciones.

Negación

Para Prolog todo lo que no se pueda demostrar como verdadero, es falso.

De esta manera el predicado `\+` implementa la **negación por falla**. Esto puede llevar a resultados inesperados, dado que la negación por falla no es equivalente a la negación lógica (dada la regla `legal(X) :- \+ ilegal(X).`, la consulta `?- legal(X)` no puede ser usada para listar todos los valores de `X` que son legales).

Aritmética

El operador `is` permite escribir predicados que involucran expresiones aritméticas, pero tiene algunas limitaciones. Se recomienda usar la biblioteca `clp(fd)` (`:- use_module(library(clpfd)).`).

- Comparaciones: `A #= B`, `A #< B`, `A #> B`, etc.
- Operaciones: `A + B`, `A - B`, etc.

Listas

Los predicados que involucran listas suelen ser recursivos. Algunos predicados predefinidos para operar con listas:

- `member(X, L)`: Verdadero si `X` es un elemento de la lista `L`.
- `append(L1, L2, L3)`: Verdadero si `L3` es la concatenación de `L1` y `L2`.
- `reverse(L, R)`: Verdadero si `R` es la lista `L` invertida.
- `permutation(L1, L2)`: Verdadero si `L2` es una permutación de `L1`.

Cálculo Lambda

Consiste en escribir expresiones lambda con una sintaxis formal, y aplicar reglas de reducción para transformarlas.

Una expresión lambda puede ser:

- Una **variable**: $x, y, z, \dots$
- Una **abstracción**: $(\lambda x.M)$ donde M es una expresión lambda.
- Una **aplicación**: $(M N)$ donde M y N son expresiones lambda.

El Cálculo Lambda incorpora dos simplificaciones para operar con funciones:

- Las funciones son anónimas:
 - $\text{suma_cuadrados}(x, y) = x^2 + y^2$ equivalente a $(x, y) \mapsto x^2 + y^2$
- Las funciones son de un solo argumento (**currificación**):
 - $(x, y) \mapsto x^2 + y^2$ equivalente a $x \mapsto (y \mapsto x^2 + y^2)$

Booleanos

```
True  = λx y.x
False = λx y.y

Not  = λp.p False True
And  = λp q.p q False
Or   = λp q.p True q
If   = λp q r.p q r
```

Numerales de Church

```
0 = λf x.x
1 = λf x.f x
2 = λf x.f (f x)
3 = λf x.f (f (f x))

Succ = λn.λf.x.f (n f x)
Pred = λn.λf.λx.n (λg.λh.h (g f)) (λu.x) (λu.u)

Add = λm n.λf x.m f (n f x)
Sub = λm n.n Pred m
Mul = λm n.λf x.m (n f) x
Pow = λm n.λf x.n m f x

IsZero := λn.n (λx.False) True
Leq := λm n.IsZero (Sub m n)
```

Pares:

```
Pair = λx y.λs.s x y
Nil := Pair True True

First = λp.p True
Second = λp.p False
```

Listas:

La lista $[a\ b\ c]$ se representa como
 $(\text{False}, (a, (\text{False}, (b, (\text{False}, (c, \text{Nil}))))))$

```
Null = IsEmpty = First
Cons = λh.λt.Pair False (Pair h t)
Head = λz.First (Second z)
Tail = λz.Second (Second z)

Así armamos la lista [a b c]:
(Cons a (Cons b (Cons c Nil)))
```

Convenciones

Para simplificar la notación:

1. Se pueden omitir los paréntesis externos:

$$M\ N \equiv (M\ N)$$

2. Las aplicaciones se asocian a la izquierda:

$$M\ N\ P \equiv ((M\ N)\ P)$$

3. El cuerpo de la abstracción se extiende todo lo posible hacia la derecha:

$$\lambda x.M\ N \equiv \lambda x.(M\ N)$$

4. Se pueden contraer múltiples abstracciones lambda:

$$\lambda x.\lambda y.\lambda z.N \equiv \lambda x\ y\ z.N$$

5. Cuando todas las variables son de una única letra, se pueden omitir los espacios:

$$MNP \equiv M N P$$

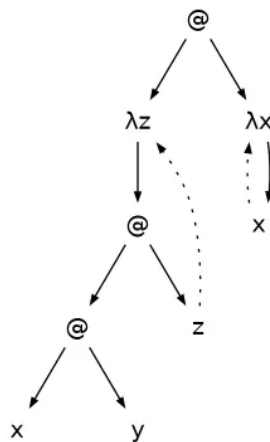
Ejemplo:

$(((\lambda x.(\lambda y.(y x))) a) b)$
 1. $(((\lambda x.(\lambda y.(y x))) a) b$
 2. $(\lambda x.(\lambda y.(y x))) a b$
 3. $(\lambda x.\lambda y.y x) a b$
 4. $(\lambda x y.y x) a b$
 5. $(\lambda x y.y x)ab$

Variables libres y ligadas

Una abstracción $\lambda x.M$ **liga** las variables x que aparecen en el cuerpo M . Todas las demás variables en M son **libres**.

Ejemplo: $(\lambda z.x y z) (\lambda x.x)$



Reglas de reducción

Conversión Alfa (α): cambiar variables ligadas

- En la abstracción $\lambda x.M$, se puede cambiar x por cualquier otra variable que no aparezca libre en M .
- Por ejemplo, $\lambda x.x y = \alpha \lambda z.z y$.

Reducción Beta (β): aplicar una β -redex

- Una β -redex es una aplicación de la forma $(\lambda x.M) N$.
- Por ejemplo, $(\lambda u.u z u) a = \beta a z a$.

Reducción Eta (η): Reducir una η -redex

- Una η -redex es una expresión de la forma $\lambda x.M x$ donde x no aparece libre en M .
- Según la regla η , $\lambda x.M x = \eta M$

Forma normal

Una expresión lambda está en:

- Forma β -normal: si no contiene ninguna β -redex.
 - Ejemplo: $z (\lambda x.y x) w$

- Forma β - η -normal: si no contiene ninguna β -redex ni η -redex.
 - Ejemplo: $z (\lambda x.x) w$
- Forma normal de cabecera: si no hay una β -redex en la cabeza de la expresión.
 - Ejemplo: $z ((\lambda x.x) w)$

Para "evaluar" una expresión lambda, se aplican las **reglas de reducción** repetidamente hasta llegar a una forma normal. La forma normal de una expresión, si existe, es **única**.

Estrategias

- **Orden normal:** siempre se reduce la β -redex más **externa** y a la izquierda.
 - Si la expresión tiene forma normal, esta estrategia la encuentra.
- **Orden aplicativo:** siempre se reduce la β -redex más **interna** y a la izquierda.
 - Esta estrategia puede no terminar, incluso si la expresión tiene forma normal.
- **Call by value:** Como el orden aplicativo, pero no se aplican reducciones dentro de abstracciones.
 - Los argumentos de una función se evalúan antes de llamar a la función.
- **Call by name:** Como el orden normal, pero no se aplican reducciones dentro de abstracciones.
 - Las funciones reciben expresiones sin evaluar, y las evalúan solo cuando las necesitan.

Combinadores

Una expresión lambda que no tiene variables libres es un combinador.

La lógica combinatoria es un modelo de computación equivalente al cálculo lambda pero más simple, ya que usa combinadores en lugar de abstracciones.

Recursión

Un combinador de punto fijo es una función que recibe una función f y devuelve un punto fijo; es decir un valor p tal que $f p = p$.

Se utiliza esto ya que en cálculo lambda las funciones no tienen nombre como para llamarlas como en los lenguajes tradicionales.

Combinador Y

Permite escribir funciones recursivas:

$$Y = \lambda f.(\lambda x.f (x x)) (\lambda x.f (x x))$$

Verificación:

$$\begin{aligned} Y &= (\lambda x.g (x x)) (\lambda x.g (x x)) \\ &= g ((\lambda x.g (x x)) (\lambda x.g (x x))) \\ &= g (Y g) \end{aligned}$$

Produce stack overflow en lenguajes como C y Python.

Combinador Z

Variante que resuelve el problema de stack overflow del combinador Y:

$$Z = \lambda f.(\lambda x.f (\lambda v.x x v)) (\lambda x.f (\lambda v.x x v))$$

Programación funcional (Clojure)

En la programación funcional los programas se construyen mediante la **aplicación y composición de funciones**.

Es un paradigma **declarativo** en el que el cuerpo de las funciones contiene **expresiones** en lugar de instrucciones.

Ventajas:

- **Código reutilizable:** La combinación de funciones puras y de orden superior suele resultar en un código más modular y reutilizable.
- **Más fácil de razonar:** Al evitar el estado compartido y los datos mutables, el comportamiento de una función depende únicamente de sus entradas.
- **Más fácil de depurar y probar:** Las funciones puras son triviales de probar; se proporciona una entrada y se verifica una salida.
- **Menos código:** Los lenguajes funcionales suelen ser más expresivos y concisos, lo que reduce la cantidad de código necesario para lograr el mismo resultado.
- **Concurrencia más segura:** Los datos inmutables son inherentemente seguros para operaciones concurrentes, eliminando categorías enteras de errores comunes como condiciones de carrera y bloqueos.

Literales: [🔗](#)

```
1
nil
true
un-simbolo
:un-keyword
"una cadena"
["un" "vector" "de" "cadenas"]
("una" "lista" "de" "cadenas")
{:tipo "mapa" :clave "valor"}
#{ "un conjunto" "de cadenas" }
```

Las listas se interpretan como una **aplicación de función** (funcion arg1 arg2 ...): [🔗](#)

```
(+ 1 2) ; => 3
(str "Hola" " mundo!") ; => "Hola mundo!"
(1 2 3) ; => error
'(1 2 3) ; => (1 2 3)
```

Formas especiales: [🔗](#)

```
(def pi 3.14159)

(def suma (fn [a b] (+ a b)))

(defn suma [a b] (+ a b)) ; equivalente

(def suma #(+ %1 %2)) ; función anónima

(if (> 3 2)
  (println "Tres es mayor que dos")
  (println "Tres no es mayor que dos"))

(let [x 10 y 20]
  (println "La suma es:" (+ x y)))
```

Funciones puras

Aquella que cumple con:

- Comportamiento **determinístico**: Para argumentos idénticos siempre produce valores de retorno idénticos.
- No produce efectos colaterales.

Un programa funcional puro opera exclusivamente con **datos inmutables**. En lugar de alterar valores existentes, se crean copias alteradas y se preserva el original.

✗ Impura (no es determinística):

```
(defn tirar-moneda []  
  (if (= (rand-int 2) 0)  
    :cara  
    :ceca))
```

✓ Pura :

```
(defn sumar [a b]  
  (+ a b))
```

✗ Impura (produce efectos colaterales):

```
(defn saludar [nombre]  
  (println "Hola," nombre))
```

Transparencia referencial

Una expresión es referencialmente transparente si puede ser **reemplazada por su valor** sin cambiar el comportamiento del programa.

Las funciones puras son referencialmente transparentes, lo que permite razonar acerca del código y modificarlo sin efectos inesperados.

Funciones como ciudadanos de primera clase

Las funciones pueden ser tratadas como cualquier otro valor: pueden ser asignadas a variables, pasadas como argumentos y devueltas como resultados.

Ejemplo: La función anónima es una clausura (closure) que captura el valor de `x` del entorno donde fue definida.

Funciones de orden superior

Aquella que puede recibir funciones como argumentos o devolver funciones como resultado (ej: `map`, `filter` y `reduce`).

```
(defn dos-veces [f x] (f (f x)))  
  
(println (dos-veces inc 5)) ; 7  
(println (dos-veces #(str % "!") "Hola")) ; "Hola!!"
```

Recursión

En los lenguajes funcionales, la iteración (ciclos) se logra mediante la recursión.

Si la llamada recursiva está en posición de cola, se puede usar `recur`. De esta forma, el compilador puede optimizar la llamada recursiva y evitar el crecimiento del stack.

```
(defn _buscar [v x i]  
  (if (>= i (count v))  
    nil  
    (if (= (nth v i) x)  
      i  
      (_buscar v x (inc i)))))  
  
(defn buscar [v x]  
  (_buscar v x 0))
```

```
(defn buscar [v x]  
  (loop [i 0]  
    (if (>= i (count v))  
      nil  
      (if (= (nth v i) x)  
        i  
        (recur (inc i))))))
```

Evaluación perezosa

Estrategia de evaluación de expresiones que **retrasa la evaluación** de una sub-expresión hasta que su valor sea realmente necesario.

En Clojure, la mayoría de las funciones que operan sobre secuencias son perezosas.

Estructuras de datos funcionales

Las estructuras de datos funcionales, o estructuras de datos **persistentes**, son aquellas que siempre preservan la versión anterior ante cualquier modificación.

Son efectivamente **inmutables**.

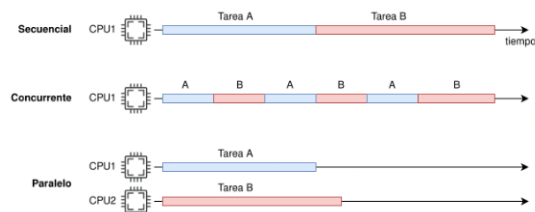
Programación concurrente

Concurrencia es la habilidad de lidiar con múltiples tareas que pueden estar en progreso al mismo tiempo (pero no necesariamente en ejecución).

Paralelismo es la habilidad de ejecutar múltiples tareas en forma simultánea. Requiere soporte de hardware (múltiples CPUs o cores).

Los programas generalmente se pueden **categorizar** en dos tipos:

- Un programa es I/O bound cuando hace uso intensivo de operaciones de entrada/salida. Ejemplos: servidores, bases de datos, interfaces de usuario.
 - Suelen beneficiarse más de la **concurrencia**.
- Un programa es CPU bound cuando hace uso intensivo del CPU. Ejemplos: simuladores, videojuegos, procesamiento de imágenes, criptografía, IA.
 - Suelen beneficiarse más del **paralelismo**.



Procesos

Unidades de ejecución independientes y aisladas, con su **propio espacio de memoria**. Son manejados por el sistema operativo.

El **scheduler** es el componente del SO que asigna tiempo de CPU a cada proceso (y en el caso de sistemas multicore, asigna los diferentes cores).

Para comunicarse entre sí, los procesos requieren mecanismos de IPC (comunicación interproceso) provistos por el SO, como **pipes** o **sockets**.

Threads

Son unidades de ejecución dentro de un proceso que comparten el mismo espacio de memoria.

Pueden ser manejados por el sistema operativo o por la propia aplicación (dependiendo del lenguaje y la biblioteca de threads utilizada).

Estados:

- New
- Ready
- Running
- Terminated

Estados intermedios:

- Timed Waiting
- Waiting
- Blocked

Condiciones de carrera

Ocurre cuando el orden de las operaciones realizadas por dos o más hilos afecta el resultado final, a menudo provocando bugs.

Una función o objeto es **thread-safe** si puede ser utilizado por múltiples hilos sin condiciones de carrera.

Locks

Mecanismo de sincronización que permite a los hilos coordinar el acceso a recursos compartidos, previniendo condiciones de carrera.

Deadlocks

Ocurre cuando dos o más hilos están bloqueados permanentemente, esperando por un recurso que nunca será liberado.

Una solución es asegurar un orden global para adquirir múltiples candados.

```
final Object lock1 = new Object();
final Object lock2 = new Object();

Thread hiloA = new Thread() -> {
    synchronized (lock1) {
        System.out.println("Hilo A: Adquirió lock 1");
        try { Thread.sleep(100); } catch (InterruptedException e) {}
        System.out.println("Hilo A: Esperando por lock 2");
        synchronized (lock2) { System.out.println("Hilo A: Adquirió lock 2"); }
    }
};

Thread hiloB = new Thread() -> {
    synchronized (lock1) { // Cambiado lock2 por lock1
        System.out.println("Hilo B: Adquirió lock 1");
        try { Thread.sleep(100); } catch (InterruptedException e) {}
        System.out.println("Hilo B: Esperando por lock 2");
        synchronized (lock2) { System.out.println("Hilo B: Adquirió lock 2"); }
    }
};

hiloA.start(); hiloB.start();
```

Variables de condición

Permite a un hilo suspender su ejecución hasta que se cumpla una condición específica.

```

public class CasillaDeCorreo {
    private Object paquete;
    private boolean vacio;

    public synchronized void depositar(Object p) throws InterruptedException {
        while (!vacio) {
            wait(); // Bloquea el hilo hasta que la casilla esté vacía
        }
        paquete = p;
        vacio = false;
        notify(); // Despierta a uno de los hilos que esperan
    }

    public synchronized Object retirar() throws InterruptedException {
        while (vacio) {
            wait(); // Bloquea el hilo hasta que haya un paquete
        }
        Object p = paquete;
        paquete = null;
        vacio = true;
        notify(); // Despierta a uno de los hilos que esperan
        return p;
    }
}

```

Variables atómicas

Una transacción atómica es una serie indivisible e irreducible de operaciones, de modo que o todas ocurren, o ninguna ocurre.

Computación asíncrona

Una función **sincrónica** bloquea el hilo que la invoca hasta que la operación finaliza.

Una función **asíncrona** retorna inmediatamente, permitiendo que el hilo invocante continúe su ejecución.

Modelos de concurrencia alternativos (Clojure)

```

(def candado (Object.))

(def hilo1 (Thread. (fn []
    (println "1: esperando candado")
    (locking candado
        (println "1: candado adquirido")
        (Thread/sleep 3000)
        (println "1: candado liberado")))))

(def hilo2 (Thread. (fn []
    (println "2: esperando candado")
    (locking candado
        (println "2: candado adquirido")
        (Thread/sleep 1000)
        (println "2: candado liberado")))))

(run! #(.start %) [hilo1 hilo2])
(run! #(.join %) [hilo1 hilo2])

```

Promise

- `(promise)` crea una nueva promesa.
- `(deliver p v)` asigna el valor `v` a la promesa `p`.
- `(deref p)` o `@p` bloquea hasta que la promesa tenga un valor y lo devuelve.

```

(defn tarea-muy-costosa [n]
  (Thread/sleep 3000)
  (+ n 10))

(def p (promise))

(.start (Thread. (fn []
                  (let [r (tarea-muy-costosa 1)]
                    (deliver p r)
                    (println "deliver")))))

(println "deref 1:")
(println @p)

(println "deref 2:")
(println @p)

```

Future

- `(future expr)` evalúa la expresión en un hilo separado.
- `(deref f)` o `@f` bloquea hasta que el resultado esté disponible y lo devuelve.

```

(defn tarea-muy-costosa [n]
  (Thread/sleep 3000)
  (+ n 10))

(def f1 (future (tarea-muy-costosa 1)))
(def f2 (future (tarea-muy-costosa 2)))
(def f3 (future (tarea-muy-costosa 3)))
(println "Hilos lanzados")

(println "Deref 1:")
(println (map deref [f1 f2 f3]))

(println "Deref 2:")
(println (map deref [f1 f2 f3]))

; necesario para evitar una espera de 1 minuto antes de
; finalizar el proceso:
(shutdown-agents)

```

pcalls, pvalues y pmap

- `(pcalls f1 f2 f3)` : Ejecuta las funciones en hilos separados.
- `(pvalues expr1 expr2 expr3)` : Evalúa las expresiones en hilos separados.
- `(pmap f coll)` : Similar a `map` , pero ejecuta las aplicaciones de `f` en hilos separados.

```

(defn tarea-muy-costosa [n]
  (Thread/sleep 3000)
  (+ n 10))

(println (time (doall (map tarea-muy-costosa (range 4)))))
;; "Elapsed time: 12001.447199 msecs"
;; (10 11 12 13)

(println (time (doall (pmap tarea-muy-costosa (range 4)))))
;; => "Elapsed time: 3012.111684 msecs"
;; => (10 11 12 13)

(shutdown-agents)

```

Delay

- `(delay expr)` crea un objeto de tipo `Delay` , que asegura que la expresión se evalúa una única vez la primera vez que es desreferenciada.
- `(deref d)` o `@d` obtiene el resultado de la evaluación de la expresión, bloqueando si aún no se ha terminado de evaluar.

```

(defn tarea-muy-costosa [n]
  (println "evaluando!")
  (Thread/sleep 3000)
  (+ n 10))

(def d (delay (tarea-muy-costosa 1)))
(println "Delay creado")

(println (time @d))
(println (time @d))

(shutdown-agents)

```

Refs & STM

- `(ref valor-inicial)` crea un ref con el valor inicial.
- `(deref r)` o `@r` devuelve el valor actual del ref.
- `(dosync expr)` ejecuta la expresión en una transacción atómica.
- `(alter r f)` dentro de una transacción, actualiza el valor del ref

```

(def prox-numero-libre (ref 0))
(def total-recaudado (ref 0))

(defn reservar-numero [precio]
  (dosync (let [n @prox-numero-libre
               t (alter total-recaudado + precio)]
            (alter prox-numero-libre inc)
            [n t])))

(dotimes [_ 8]
  (.start (Thread. (fn []
                    (Thread/sleep (rand-int 300))
                    (let [[n t] (reservar-numero (rand-int 5000))]
                      (locking *out* (println (format "número reservado: %d, total recaudado: %d" n t)))
                      (recur)))))))

```

Atoms

- `(atom valor-inicial)` crea un átomo con el valor inicial.
- `(deref a)` o `@a` devuelve el valor actual del átomo.
- `(swap! a f)` actualiza en forma atómica el valor del átomo, y devuelve el valor anterior.
- `(swap-vals! a f)` similar a `swap!`, pero devuelve una secuencia con el valor anterior y el nuevo valor.

```

(def estado-rifa (atom {:prox-numero-libre 0
                        :total-recaudado 0}))

(defn reservar-numero [precio]
  (let [actualizar #(-> %
                        (update :prox-numero-libre inc)
                        (update :total-recaudado + precio))
        [viejo nuevo] (swap-vals! estado-rifa actualizar)]
    [(viejo :prox-numero-libre) (nuevo :total-recaudado)]))

(dotimes [_ 8]
  (.start (Thread. (fn []
                    (Thread/sleep (rand-int 300))
                    (let [[n t] (reservar-numero (rand-int 5000))]
                      (locking *out* (println (format "número reservado: %d, total recaudado: %d" n t)))
                      (recur)))))))

```

Agents

- `(agent valor-inicial)` crea un agente con el valor inicial.
- `(deref a)` o `@a` devuelve el valor actual del agente.
- `(send a f)` envía una función `f` para actualizar el valor del agente en otro hilo.
- `(add-watch a k f)` ejecutará `f` cada vez que el valor del agente cambie.

```
(require '[clojure.pprint :as p])

(def rifa (agent {:proximo-numero-libre 0
                  :reservas []}))

(defn reservar-numero [persona precio]
  (send rifa (fn [estado]
               (let [n (:proximo-numero-libre estado)
                     nueva-reserva {:persona persona
                                     :numero n
                                     :precio precio}]
                 (-> estado
                     (update :proximo-numero-libre inc)
                     (update :reservas conj nueva-reserva))))))

(dorun (pmap #(reservar-numero % (rand-int 5000))
              ["Ana" "Beto" "Carla" "Diego" "Eva" "Fabián" "Gisela" "Hugo"]))

(await rifa)
(pprint @rifa)
(shutdown-agents)
```

Reducers

Las funciones de la biblioteca `clojure.core.reducers` permiten procesar grandes cantidades de datos mediante el patrón **fork/join**.

```
(require '[clojure.core.reducers :as r])

(def numeros (vec (range 100000)))

(println (r/fold 512 ;; tamaño de la partición
                +    ;; función para combinar el resultado de dos particiones
                +    ;; función para reducir una partición
                (r/filter odd? numeros)))
```

Communicating Sequential Processes (CSP)

Modelo de concurrencia basado en procesos livianos que se comunican mediante el paso de mensajes a través de canales.

La biblioteca `core.async` es una implementación de CSP en Clojure, inspirada en las goroutines y channels de Go.

- `(chan)` crea un canal.
- `(>! c v)` encola `v` en el canal `c` (bloquea si el buffer del canal está lleno).
- `(<! c)` desencola del canal `c` (bloquea si no hay nada para leer).
- `(close! c)` cierra el canal `c` (`<!` devolverá `nil`).
- `(go ...)` ejecuta el cuerpo en forma asíncrona en un go block. Devuelve un canal que contendrá el resultado.

Corutinas

Función que puede ser pausada y reanudada, permitiendo la concurrencia cooperativa.

Originalmente las corutinas en Python se usaban para implementar generadores, que son una forma conveniente de crear iteradores.

Para trabajar con corutinas de forma más práctica, recientemente se agregó al lenguaje la sintaxis

`async` / `await` y el módulo `asyncio` .

```
def co1():
    print("co1 haciendo cosas...")
    yield
    print("co1 haciendo cosas...")
    yield

def co2():
    print("co2 haciendo cosas...")
    yield
    print("co2 haciendo cosas...")
    yield
```

```
def scheduler(corutinas):
    cola = corutinas[:]
    while cola:
        try:
            co = cola.pop(0)
            co.send(None)
            cola.append(co)
        except StopIteration:
            pass
    scheduler([co1(), co2()])
```